

Project 04 — E-Commerce Store

Course: Web Technologies | **Format:** Lab Exam | **Duration:** 3–4 hours (TBA)

Application Overview

A product marketplace where customers browse and filter products, add them to a session-based cart, and complete checkout, while an admin manages the product catalogue and processes orders. Customers can also leave ratings and reviews. Four students build separate, independently testable features that share one database.

Team Structure

Task	Feature	Student
Task 1	User Authentication & Profile	Student 1
Task 2	Product & Category Management (<i>Admin</i>)	Student 2
Task 3	Shopping Cart & Checkout (<i>Customer</i>)	Student 3
Task 4	Order Management & Product Reviews	Student 4

Shared Database Schema

Table	Key Columns
users	id, name, email, password_hash, phone, role (customer/admin), shipping_addresses (JSON), remember_token, created_at
categories	id, parent_id (nullable), name
products	id, category_id, name, description, price, stock_qty, primary_image_path, is_available, created_at
orders	id, user_id, shipping_address, payment_method, total_amount, status (Pending/Processing/Shipped/Delivered/Cancelled), created_at
order_items	id, order_id, product_id, quantity, unit_price
reviews	id, product_id, user_id, rating (1–5), review_text, created_at

Cart is stored in `$_SESSION['cart']` as `[product_id => quantity]` — no separate table.

Teammates may share only the database schema among themselves. No other code can be shared!

Global Technical Requirements (*all students*)

- **PHP MVC structure:** `controllers/`, `models/`, `views/`, `config/`
- Hash passwords with `password_hash()` / verify with `password_verify()`
- **PDO/mysqli + prepared statements** for every query
- **Server-side validation** on every form; show inline error messages
- `session_start()` on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return `Content-Type: application/json`

- Uploaded files go to `public/uploads/`; validate MIME type + size server-side
 - Do **not** drop or alter the shared schema tables
-

Task 1 — User Authentication & Profile

What You Build

The identity layer: registration, login with persistent session and cookie, and profile management for customers and admins.

Requirements

1. **Registration** — Form collects name, email, optional phone, and password (≥ 8 chars). Validate email uniqueness. Hash password. New accounts default to `role = 'customer'`. Redirect to login on success.
2. **Login & Remember Me** — Login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. "Remember Me" checkbox stores a hashed token in `users.remember_token` and writes a 30-day cookie; on next visit PHP reinstates the session. Admins are redirected to the admin dashboard; customers to the product catalogue.
3. **Customer Profile** (*session-gated*) — Customer can update name, email, phone, and up to two saved shipping addresses (stored as a JSON array in `users.shipping_addresses`). Password change requires current password verification.
4. **Admin Gate** — All admin-facing pages (Tasks 2 and 4 admin views) must begin with `if ($_SESSION['role'] !== 'admin') { redirect('/login'); }`. Provide a shared `require_admin()` helper function in `config/helpers.php`.

Key Outputs

`users` rows; `$_SESSION['user_id']` and `$_SESSION['role']`; `require_admin()` helper; saved shipping addresses for Task 3's checkout pre-fill.

Task 2 — Product & Category Management (*Admin*)

What You Build

The admin product catalogue: nested category management, product CRUD with primary image upload, and a live availability toggle.

Requirements

1. **Category CRUD** — Categories support one level of nesting (`parent_id`). Create/edit form includes a "Parent Category" dropdown (nullable). Delete blocks if child categories or products reference the category.
2. **Product CRUD** — Form collects: name, description, price (positive), stock quantity, category (dropdown — shows hierarchy), and primary image upload (JPEG/PNG $\leq$ 3 MB → `public/uploads/products/`). Edit pre-fills all fields. Delete is blocked if the product has existing order items.
3. **Stock Alert** — The admin product listing table highlights rows where `stock_qty  $\leq$  5` with an amber background. Each row also shows the product's average star rating (a subquery or JOIN on `reviews`) once Task 4 populates that table.
4. **Availability AJAX Toggle** — Each product row has an In Stock / Out of Stock badge. Clicking it PATCHes `/api/products/{id}/availability` ; PHP flips `is_available` ; JS swaps the badge class and text in-place without a page reload.

Key Outputs

`categories` and `products` rows for Student 3 to browse; `primary_image_path` that Task 3's product detail page displays.

Task 3 — Shopping Cart & Checkout (*Customer*)

What You Build

The complete customer purchase flow: product browsing with AJAX filters, a session-backed AJAX cart, and checkout that writes an order.

Requirements

1. **Product Catalogue** — Grid of available products (`is_available = 1`). AJAX keyword search bar calls `GET /api/products/search?q=...` on each keystroke. Category filter dropdown fires `GET /api/products?category_id=...` via AJAX. Both re-render the product grid in-place. Clicking a product opens its detail page, which shows the primary image, description, price, stock status, and average rating.
2. **Add to Cart (AJAX)** — "Add to Cart" button POSTs `{product_id}` to `/api/cart/add`; PHP increments quantity in `$_SESSION['cart']` (capped at available stock) and returns the new total item count; JS updates the navbar cart icon via DOM.
3. **Cart Page** — Lists cart items with name, unit price, `+` / `-` quantity buttons, and Remove. Each button fires a single AJAX call to `/api/cart/update` or `/api/cart/remove`; PHP adjusts `$_SESSION['cart']`, returns new line total and grand total; JS updates only the totals section — no full reload.
4. **Checkout & Confirmation** — Checkout form shows saved shipping addresses as radio buttons (pre-selected from profile) plus a "Use a new address" option. Payment method radio (Cash / Card). On submit PHP validates cart is non-empty and stock is sufficient, writes `orders` + `order_items` rows, decrements `products.stock_qty`, clears `$_SESSION['cart']`, redirects to confirmation page with order ID and summary.

Key Outputs

`orders` and `order_items` rows that Student 4 reads and manages.

Task 4 — Order Management & Product Reviews

What You Build

Customer order history, admin order fulfilment with a REST endpoint, and a product review system with star ratings.

Requirements

1. **Customer "My Orders"** — Lists all orders for the logged-in user, newest first. Each row shows order ID, date, total, and a colour-coded status badge. Clicking expands an itemised order-detail view.

2. **Admin Order Management** — Admin-only page lists all orders with filters (status dropdown, date range). Each row's status can be changed via a dropdown; changing it fires `PUT /api/orders/{id}` with `{status}`; PHP updates the DB and returns `{"ok":true}`; JS updates the badge in-place. Allowed status sequence: Pending → Processing → Shipped → Delivered; or any state → Cancelled.
3. **Product Review** — On the customer's order-detail page, products in a Delivered order show a "Leave a Review" form (1–5 star rating via radio buttons + text). Submitting via AJAX POSTs to `/api/reviews`; PHP enforces one review per user per product (unique constraint); the new review appears below the form immediately.
4. **Average Rating Display** — The product detail page (built in Task 3) shows the average star rating and a list of all reviews fetched via `GET /api/products/{id}/reviews`. Average is also shown on the product cards in the catalogue grid (as a static HTML value computed server-side on page load).

Key Outputs

Complete post-purchase loop demonstrating the full application.

Submission Checklist

- ☐ All pages load without PHP errors
- ☐ Cart correctly caps quantity at available stock
- ☐ AJAX calls handle both success and error responses
- ☐ One review per user per product is enforced at DB level